Health AI

1.Introduction

- Project Title: Health AI – An Intelligent Healthcare Assistant
- Team Member: Fathima Arshin A
- Team Member: Bhavana R
- Team Member: Afreen Fathima S
- Team Member: Thasleema Parveen P

2.Project Overview

Purpose:

. The purpose of Health AI is to provide intelligent healthcare assistance by collecting patient inputs such as symptoms, age, and medical history, and generating appropriate remedies, medication guidance, and vitamin deficiency detection.
The system leverages artificial intelligence, natural language processing, and machine learning models to analyze patient-reported data. Based on the analysis, it provides personalized health advice, lifestyle recommendations, and early alerts for potential health issues.
By serving both patients and healthcare providers, Health AI bridges the gap between medical consultation and preventive healthcare, making healthcare more accessible, efficient, and proactive.

Features:

Symptom Checker:

Key Point: Patient input analysis
Functionality: Asks users for symptoms, age, and medical history to suggest possible conditions and remedies.

Medication Guidance:

Key Point: Treatment recommendations
Functionality: Provides basic medication suggestions (OTC medicines, precautions) and prompts users to consult a doctor when necessary.

Vitamin Deficiency Detection:

Key Point: Preventive healthcare
Functionality: Identifies possible vitamin or nutrient deficiencies based on reported symptoms (e.g., fatigue → Vitamin D, hair loss → Biotin).

Health Tips & Remedies:

Key Point: Personalized wellness support
Functionality: Suggests home remedies, diet adjustments, and lifestyle practices to improve health outcomes.

Medical Report Summarization:

Key Point: Simplified understanding
Functionality: Summarizes uploaded lab reports or prescriptions for easy patient interpretation.

Anomaly Detection:

Key Point: Early warning system
Functionality: Detects unusual patterns in health history and alerts the user about potential risks.

Multimodal Input Support:

Key Point: Flexible health data handling
Functionality: Accepts patient records, medical prescriptions, and lab reports in text/PDF for analysis.

User-Friendly Interface:

Key Point: Accessibility
Functionality: Provides an interactive chatbot/dashboard for users to input symptoms and receive recommendations.

3. Architecture:

User Input Layer (Gradio Interface):

Patients interact with the system via a chatbot-like interface built using Gradio.
Inputs include symptoms, age, and medical history.

AI Model Layer (IBM Granite Model):

The project uses IBM Granite models (such as granite-3.2-2b-instruct) hosted on Hugging Face.
The model processes user queries and generates remedies, medication guidance, and vitamin deficiency detection.

Execution Layer (Google Colab Environment):

The application is deployed in Google Colab, utilizing T4 GPU for fast execution.

Required Python libraries (transformers, torch, gradio) are installed and executed in Colab.

Data Flow:

Input (symptoms, age, medical history) → Model Processing (Granite LLM + rules/ML) → Output (remedies, medication guidance, vitamin deficiency alerts).

Version Control (GitHub):

Final code is exported from Colab and uploaded to GitHub for version control, collaboration, and future enhancements.

4. Setup Instructions:

- Basic knowledge of Python programming.
- Familiarity with Gradio framework.
- Access to IBM Granite models on Hugging Face.
- Google Colab account (with GPU access).
- GitHub account for uploading the project.

Step-by-Step Setup:
Access Project Resources:

Log in to the Naan Mudhalvan Smart Interns portal.
Enroll in the Health AI project and access the “Guided Project” section.

Select IBM Granite Model:

Visit Hugging Face.
Create an account and search for IBM Granite models.
Select a lightweight model such as granite-3.2-2b-instruct.

Run in Google Colab:

Open Google Colab.
Create a new notebook named Health AI.
Change runtime to T4 GPU (Runtime → Change runtime type → GPU).

Install dependencies:

!pip install transformers torch gradio -q
Copy and run the Health AI project code.

Test the Application:

Once the model loads, a Gradio URL will appear.
Open the link to interact with the chatbot and test remedies, medication guidance, and vitamin deficiency detection.
Upload to GitHub
Create a repository in GitHub.
Download the Colab notebook/code as .py.

5.Folder Structure

app/ – Backend logic (routers, models, API integration).

app/api/ – Modular API routes for chat, feedback, reports, document vectorization.

ui/ – Streamlit UI components.

granite_llm.py – Communication with IBM Watsonx Granite model.

document_embedder.py – Document embeddings with Pinecone.

kpi_file_forecaster.py – Forecasting trends (future scope for public health analytics).

report_generator.py – AI-generated medical/sustainability reports.

6.Running the Application

Frontend (Streamlit)

The frontend is developed using Streamlit, providing an interactive web UI with multiple modules:

- Health dashboard
- Report upload & analysis
- Chat-based health assistant
- Diet & exercise planner
- Progress tracking with charts
- Navigation is handled with the streamlit-option-menu library. Each page is modular for scalability.

Backend (FastAPI)

The backend is built with FastAPI, enabling RESTful APIs for:

- Health data analysis
- Report summarization
- Predictive analytics
- Personalized recommendations

- Citizen feedback storage
- It supports asynchronous performance and integrates with Swagger UI for testing and documentation.

7. API Documentation

POST /chat/ask – Symptom analysis & AI-generated response.

POST /upload-doc – Document upload and embedding.

GET /search-docs – Retrieve semantically similar medical references.

GET /get-eco-tips – Health/environment-related lifestyle tips.

POST /submit-feedback – Collects user/patient feedback.

6. history tracking.

8. Authentication

The project runs in an open environment for demonstration. For secure deployments, it supports:

Token-based authentication (JWT / API keys)

OAuth2 with IBM/Naan Mudalvan credentials

Role-based access (Student, Citizen, Doctor, Admin)

Planned enhancements: user sessions & health history tracking

9.User Interface

The interface is minimalist and user-friendly, focusing on accessibility for all users. It includes:

Minimalist and accessible design.

Sidebar navigation for disease prediction and treatment plan generation.

Real-time interaction with backend APIs.

PDF report generation for patient records.

The interface is minimalist and user-friendly, focusing on accessibility for all users.

10.Testing

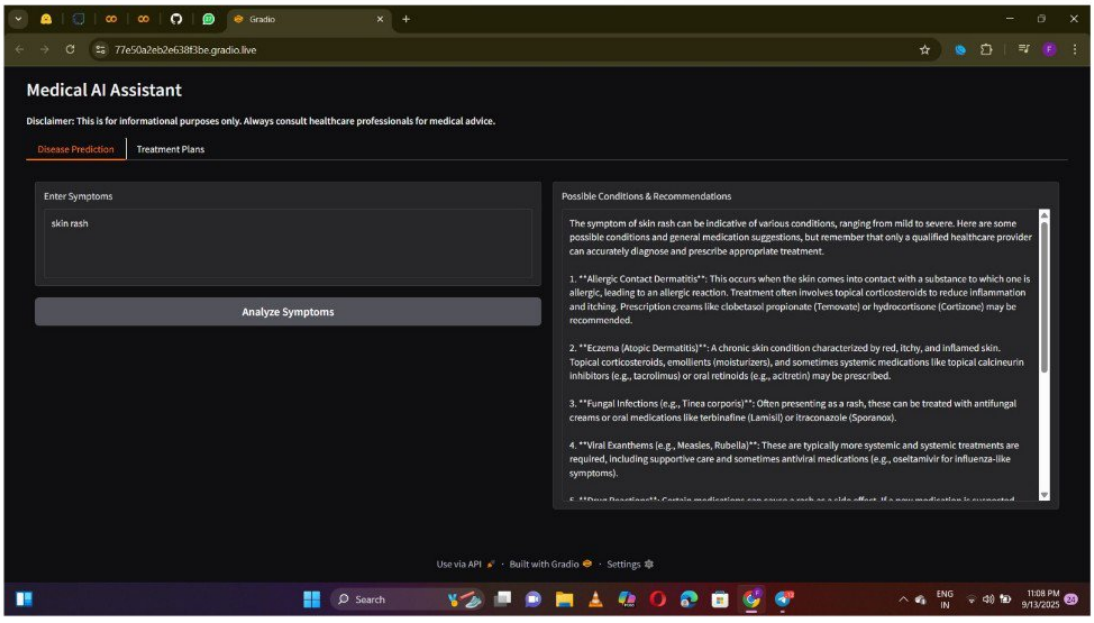
Unit Testing: Prompt engineering functions & utility scripts.

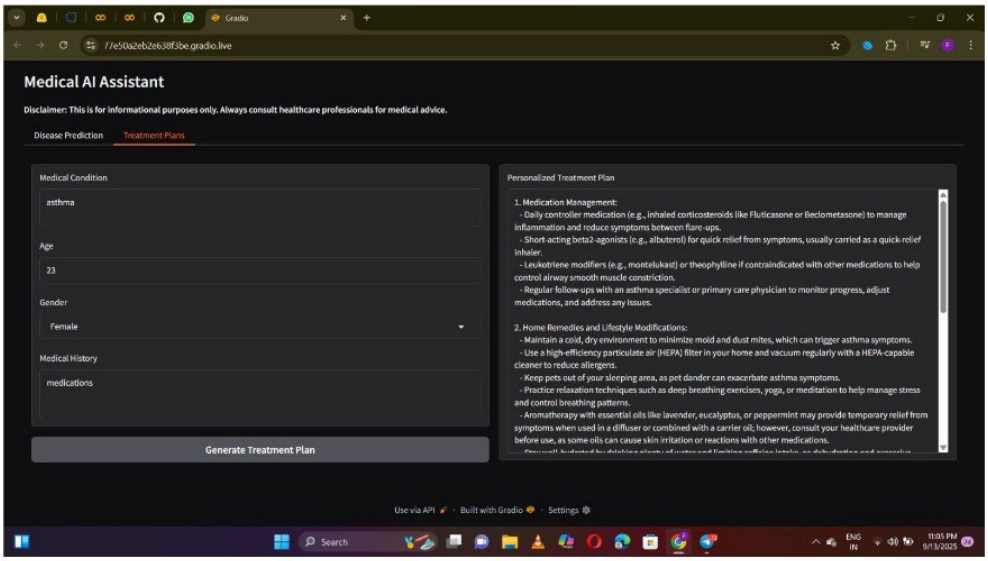
API Testing: Swagger UI, Postman, automated test scripts.

Manual Testing: File uploads, symptom entry, treatment output verification.

Edge Case Handling: Malformed inputs, large files, invalid API keys.

11.Screenshot





12.known issues

AI-generated results are informational only; cannot replace certified medical advice.

Limited dataset coverage for rare diseases.

Currently depends on internet access for model queries.

13.Future Enhancements

Integration with wearable health devices for real-time monitoring.

Multilingual support for rural and regional accessibility.

Voice-based symptom entry using speech recognition.

Expansion into mental health and nutrition modules.

Deployment as a mobile app for wider accessibility.